

API Specification — Black Fitness ระบบจองคลาส

ระบบ Black Fitness ไม่มี REST API server กลาง — Frontend เชื่อมต่อ Cloud Firestore โดยตรงผ่าน Firebase JavaScript SDK ดังนั้น "API" ในที่นี้หมายถึง **ชุดปฏิบัติการ Firestore (SDK operations)** ที่แอปเรียกใช้ พร้อม contract ของแต่ละการทำงาน (input, output, side effects, สิทธิ์, ข้อผิดพลาด)

อ้างอิงจากซอร์สโค้ดจริง ระบุไฟล์ต้นทางของแต่ละ operation

สารบัญ

1. รูปแบบการเชื่อมต่อ
2. Authentication API (LIFF)
3. Users API
4. Classes API
5. Bookings API
6. Check-ins API
7. Settings API
8. สรุปสิทธิ์ตามบทบาท
9. รูปแบบข้อผิดพลาด

1. รูปแบบการเชื่อมต่อ

```
import { db } from './firebase'
import { collection, doc, getDoc, getDocs, query, where,
  addDoc, setDoc, updateDoc, deleteDoc,
  runTransaction, increment, Timestamp } from 'firebase/firestore'
```

ทุก operation ทำงานในบริบทของ Security Rules — สิทธิ์ถูกตรวจที่ฝั่ง Firestore ตาราง operation ด้านล่างใช้สัญลักษณ์:

- **R** = read, **W** = write, **T** = transaction

2. Authentication API (LIFF)

ไม่ใช่ Firestore — เป็นการยืนยันตัวตนผ่าน LINE LIFF SDK (`stores/liff.js`)

`liff.init({ liffId })`

- **หน้าที่:** เริ่มต้น LIFF, ตรวจสอบสถานะ login
- **Output:** `isLiffReady` , `isInClient` , `initError`

`liff.getProfile()`

- **หน้าที่:** ดึงโปรไฟล์ LINE ของผู้ใช้
- **Output:** `{ userId, displayName, pictureUrl, statusMessage }`
- **Dev mode:** ค้น mock profile (กำหนดผ่าน query param ได้)

`authStore.signInWithLineUserId(lineUserId, displayName, pictureUrl)`

- **หน้าที่:** เข้าสู่ระบบ + โหลด/ตรวจโปรไฟล์จาก Firestore
 - **Side effect:** อ่าน `users/{lineUserId}` , sync รูปโปรไฟล์ถ้าเปลี่ยน
 - **Output:** userProfile + กำหนด isAdmin/isStaff/needsProfileSetup
-

3. Users API

#	Operation	Type	Path	ไฟล์
U1	getDoc	R	<code>users/{lineUserId}</code>	auth.js (signIn)
U2	getDoc	R	<code>users/{lineUserId}</code>	auth.js (refreshUserProfile)
U3	setDoc	W	<code>users/{lineUserId}</code>	auth.js (completeProfileSetup)
U4	updateDoc	W	<code>users/{lineUserId}</code>	auth.js (sync รูป)
U5	getDocs	R	<code>users</code> (ทั้ง collection)	Admin.vue (loadAllData)
U6	updateDoc	W	<code>users/{id}</code> (cooldown)	Admin.vue (setCooldown)
U7	updateDoc	W	<code>users/{id}</code> (clear cooldown)	Admin.vue (clearCooldown)
U8	updateDoc	W	<code>users/{id}</code> (membershipExpiry)	Admin.vue
U9	updateDoc	W	<code>users/{id}</code> (memberType)	Admin.vue
U10	updateDoc	W	<code>users/{id}</code> (role)	Admin.vue
U11	deleteDoc	W	<code>users/{id}</code>	Admin.vue (ลบสมาชิก)
U12	setDoc (merge)	W	<code>users/{uid}</code> (bulk import)	Admin.vue

U3 — สร้าง/บันทึกโปรไฟล์ผู้ใช้

```
await setDoc(doc(db, 'users', lineUserId), {
  ...profileData,           // nickname, firstName, lastName, nationalId,
                             // phone, birthDate, gender, healthIssues,
                             // emergencyContact, acceptTerms, acceptRisk
  profileCompleted: true,
  updatedAt: new Date()
})
```

- สิทธิ์: เจ้าของเท่านั้น (`isOwner`), role ต้องเป็น `user`
- ผลลัพธ์: เอกสารผู้ใช้ใหม่ + ออกจากสถานะ needsProfileSetup

U6 — ตั้ง Cooldown (admin)

```
await updateDoc(doc(db, 'users', userId), {
  cooldownUntil: Timestamp.fromDate(cooldownUntil), // now + N วัน 23:59:59
  cooldownReason: reason || null,
  updatedAt: new Date()
})
```

- **สิทธิ์:** admin เท่านั้น (rules ห้าม user แก้ cooldownUntil)

U8/U9/U10 — แก้สมาชิก (admin/staff)

```
// วันหมดอายุ
await updateDoc(doc(db, 'users', id), { membershipExpiry: Timestamp.fromDate(date), updatedAt: new Date() })
// แก้ประเภท
await updateDoc(doc(db, 'users', id), { memberType: newType, updatedAt: new Date() })
// บทบาท (admin เท่านั้น)
await updateDoc(doc(db, 'users', id), { role: newRole, updatedAt: new Date() })
```

U12 — นำเข้าสมาชิกจำนวนมาก (bulk import)

```
// สมาชิกใหม่
await setDoc(doc(db, 'users', uid), {
  ...fields, lineUserId: uid, createdAt, updatedAt, profileCompleted: true }, {
  merge: true })
// อัปเดตเฉพาะฟิลด์ที่เปลี่ยน
await setDoc(doc(db, 'users', uid), { ...changes, updatedAt: new Date() }, { merge: true })
```

4. Classes API

#	Operation	Type	Path / Query	ไฟล์
C1	getDocs + where	R	<code>classes</code> where <code>date == X</code>	Booking.vue (loadClasses)
C2	getDoc	R	<code>classes/{id}</code>	ClassDetail.vue
C3	getDocs	R	<code>classes</code> (ทั้งหมด)	Admin.vue (loadAllData)
C4	addDoc	W	<code>classes</code>	Admin.vue (addClass)
C5	updateDoc	W	<code>classes/{id}</code>	Admin.vue (updateClass)
C6	deleteDoc	W	<code>classes/{id}</code>	Admin.vue (deleteClass)
C7	getDocs + where (range)	R	<code>classes</code> where date ระหว่าง	weekly script
C8	getDocs + where (×3)	R	<code>classes</code> where date+time+name	weekly script
C9	addDoc	W	<code>classes</code>	weekly script
C10	updateDoc (increment)	W	<code>classes/{id}.currentBookings</code>	booking/cancel

C1 — โหลดคลาสตามวันที่

```
const snapshot = await getDocs(query(
  collection(db, 'classes'),
  where('date', '==', selectedDate) // 'yyyy-mm-dd'
))
// → [{ id, type, subtype, name, description, date, time,
//       instructor, maxCapacity, currentBookings }, ...]
```

- สิทธิ์: อ่านได้ทุกคน (`allow read: if true`)

C4 — เพิ่มคลาส (admin)

```
await addDoc(collection(db, 'classes'), {
  type, subtype,
  name: subtypeInfo.label,
  description: subtypeInfo.description,
  date, time, instructor, maxCapacity,
  currentBookings: 0,
  createdAt: new Date()
})
```

- สิทธิ์: admin เท่านั้น
- หมายเหตุ: name/description มาจาก subtype ที่เลือก (แก้เองไม่ได้)

C7/C8/C9 — สร้างคลาสรายสัปดาห์ (batch script)

```
// ดึงคลาสในช่วงสัปดาห์
getDocs(query(collection(db, 'classes'),
  where('date', '≥', fromStr), where('date', '≤', toStr)))
// ตรวจสอบซ้ำก่อนสร้าง
getDocs(query(collection(db, 'classes'),
  where('date', '==', dateStr), where('time', '==', time), where('name', '==', name)))
// สร้างถ้าไม่ซ้ำ
addDoc(collection(db, 'classes'), { ...template, date, currentBookings:0, createdAt })
```

5. Bookings API

#	Operation	Type	Path / Query	ไฟล์
B1	getDocs + where	R	<code>bookings</code> where <code>userId == X</code>	Booking.vue, History.vue
B2	runTransaction	T	<code>classes/{id}</code> + <code>bookings</code>	Booking/ClassDetail (จอง)
B3	updateDoc	W	<code>bookings/{id}</code> (ยกเลิก)	History.vue
B4	getDocs	R	<code>bookings</code> (ทั้งหมด)	Admin.vue
B5	addDoc	W	<code>bookings</code> (admin จองให้)	Admin.vue

B1 — โหลดการจองของผู้ใช้

```
const snapshot = await getDocs(query(
  collection(db, 'bookings'),
  where('userId', '==', lineUserId)
))
```

- สิทธิ์: เจ้าของ หรือ staff/admin

B2 — จองคลาส (Transaction) ★ สำคัญ

```
await runTransaction(db, async (transaction) => {
  const classSnap = await transaction.get(classRef)
  if (!classSnap.exists()) throw new Error('ไม่พบข้อมูลคลาส')

  const classData = classSnap.data()
  if (classData.currentBookings ≥ classData.maxCapacity)
    throw new Error('คลาสเต็มแล้ว')

  const bookingRef = doc(collection(db, 'bookings'))
  transaction.set(bookingRef, {
    userId, classId, className, date, time, instructor,
    status: 'confirmed',
    bookedAt: new Date(),
    canCancelUntil: new Date(`${date}T${time}:00`)
  })
  transaction.update(classRef, { currentBookings: increment(1) })
})
```

- สิทธิ์: เจ้าของ + membership valid + status='confirmed'
- **Atomic**: ตรวจสอบความจุ + สร้าง booking + เพิ่ม counter ในธุรกรรมเดียว
- **Errors**: 'ไม่พบข้อมูลคลาส', 'คลาสเต็มแล้ว'
- **เงื่อนไขก่อนเรียก (canBook)**: ไม่ติด cooldown, membership valid, ไม่ใช่ Gold, ยังไม่จองซ้ำ, เหลือ > 30 นาทีก่อนคลาส, ไม่เกินหน้าตาจองล่วงหน้า

B3 — ยกเลิกการจอง

```
await updateDoc(doc(db, 'bookings', bookingId), {
  status: 'cancelled',
  cancelledAt: new Date()
})
await updateDoc(doc(db, 'classes', classId), {
  currentBookings: increment(-1)
})
```

- สิทธิ์: เจ้าของ (เปลี่ยน confirmed → cancelled เท่านั้น)
- **เงื่อนไข (canCancel)**: ต้องเหลือ > 5 ชั่วโมงก่อนคลาสเริ่ม

- **หมายเหตุ:** ใช้ updateDoc 2 ครั้งแยกกัน (ไม่ atomic) — มี `safeCancelBooking()` แบบ transaction ใน utils/firestore.js เป็นทางเลือก

6. Check-ins API

#	Operation	Type	Path	ไฟล์
K1	getDocs	R	<code>checkins</code>	Admin.vue
K2	addDoc	W	<code>checkins</code>	Admin.vue / CheckinScanner
K3	deleteDoc	W	<code>checkins/{id}</code>	Admin.vue

K2 — บันทึกเช็คอิน (staff/admin)

```
const ref = await addDoc(collection(db, 'checkins'), {
  uid, name, memberType,
  membershipValid, // boolean - ตรวจ ณ เวลาเช็คอิน
  classId, className, classDate, classTime, instructor,
  checkedInAt: Timestamp.fromDate(now)
})
```

- **สิทธิ์:** staff/admin เท่านั้น
- **Flow:** สแกน QR/ระบุ uid → ตรวจ membership → บันทึก + แสดงผลสำเร็จ/หมดอายุ

7. Settings API

#	Operation	Type	Path	ไฟล์
S1	getDoc	R	<code>settings/classTypes</code>	Admin.vue
S2	setDoc	W	<code>settings/classTypes</code>	Admin.vue (เพิ่ม/แก้/ลบประเภท)

S1/S2 — ตั้งค่าประเภทคลาส (admin)

```
// อ่าน
const snap = await getDoc(doc(db, 'settings', 'classTypes'))
// เขียน (ทับทั้ง document)
await setDoc(doc(db, 'settings', 'classTypes'), updatedSettings)
```

- **สิทธิ์:** อ่านทุกคน, เขียนเฉพาะ admin

8. สรุปสิทธิ์ตามบทบาท

Operation	user (เจ้าของ)	staff	admin
อ่านโปรไฟล์ตัวเอง	✓	✓	✓
อ่านโปรไฟล์ผู้อื่น	✗	✓	✓
แก้โปรไฟล์ตัวเอง (ยกเว้น role/expiry/cooldown)	✓	✓	✓
ตั้ง membershipExpiry / memberType	✗	✓	✓
ตั้ง role / cooldown	✗	✗	✓
ลบสมาชิก	✗	✗	✓ *
อ่านคลาส	✓	✓	✓
เพิ่ม/แก้/ลบคลาส	✗	✗	✓
จองคลาส (ของตัวเอง)	✓	✓	✓
ยกเลิกการจอง (ของตัวเอง)	✓	✓	✓
อ่านการจองทั้งหมด	✗	✓	✓
บันทึกเช็คอิน	✗	✓	✓
ตั้งค่าประเภทคลาส	✗	✗	✓

* rules ระบุ `allow delete: if false` สำหรับ users — การลบในโค้ด Admin จะสำเร็จต่อเมื่อ rules ถูกปรับ/ยังไม่ deploy

9. รูปแบบข้อผิดพลาด

ระบบจัดการ error ผ่าน `utils/errorHandler.js` (`handleFirestoreError`) แปลงเป็นข้อความภาษาไทย และ retry สำหรับ network error (`retryOperation`)

สถานการณ์	ข้อความ/พฤติกรรม
คลาสเต็มระหว่างจอง	'คลาสเต็มแล้ว กรุณาเลือกคลาสอื่น '
ไม่พบคลาส	'ไม่พบข้อมูลคลาส '
Gold พยายามจอง	'แพ็คเกจ Gold ไม่สามารถจองคลาสได้'
network error	retry อัตโนมัติ (exponential backoff)
permission denied	แปลเป็นข้อความภาษาไทย แสดง dialog

contract การจอง/ยกเลิกใช้ SweetAlert2 แจ้งผลสำเร็จ/ผิดพลาดเป็นภาษาไทยเสมอ

เอกสารนี้สร้างจากการวิเคราะห์ซอร์สโค้ดจริง ณ เวอร์ชันปัจจุบัน